

ABSTRACT

The rapid growth of digital systems has increased malware threats, making detection essential. This project presents a Malware Detection and Analysis Dashboard developed using Flask, YARA, and Python techniques. The system allows users to upload files and scan them using YARA rules to detect malicious patterns. It also performs analysis such as entropy calculation, file type identification, and hash generation (MD5 and SHA256). Based on these parameters, a risk level is assigned. The dashboard displays results clearly and generates reports. This system improves detection accuracy and provides an efficient, user-friendly cybersecurity solution for practical use.

INTRODUCTION

In the modern digital era, the use of computer systems and the internet has grown rapidly, leading to an increase in cyber threats such as malware attacks. Malware refers to malicious software designed to damage, disrupt, or gain unauthorized access to systems and data. These attacks can result in data loss, privacy breaches, and system failures, making malware detection and analysis a crucial aspect of cybersecurity.

Traditional malware detection systems mainly rely on signature-based techniques, where known patterns are used to identify malicious files using tools like YARA. However, modern malware often uses techniques such as encryption and obfuscation to evade detection. To address these challenges, additional analysis methods like entropy calculation, file inspection, and pattern detection are required for more accurate identification.

This project develops a Malware Detection and Analysis Dashboard using Flask and Python. The system allows users to upload files, perform scanning and analysis, and view results in an interactive dashboard. It examines file properties such as size, type, entropy, and hash values, classifies risk levels, and generates reports. This approach improves detection accuracy while providing a user-friendly and efficient solution for malware analysis.

PROBLEM STATEMENT

With the rapid growth of digital technologies and internet usage, cyber threats—especially malware attacks—have increased significantly. Malware can compromise system security, steal sensitive data, and disrupt normal operations. Most existing detection systems rely on signature-based techniques using tools like YARA to identify known threats. However, these methods are not fully effective against modern malware, which often uses obfuscation and polymorphic techniques to evade detection.

Another limitation is the lack of integrated and user-friendly systems that provide both detection and detailed analysis. Many tools focus only on scanning or require technical expertise to interpret results, with limited support for visualization and reporting. Therefore, there is a need for a system that combines detection with advanced analysis and presents results through an interactive

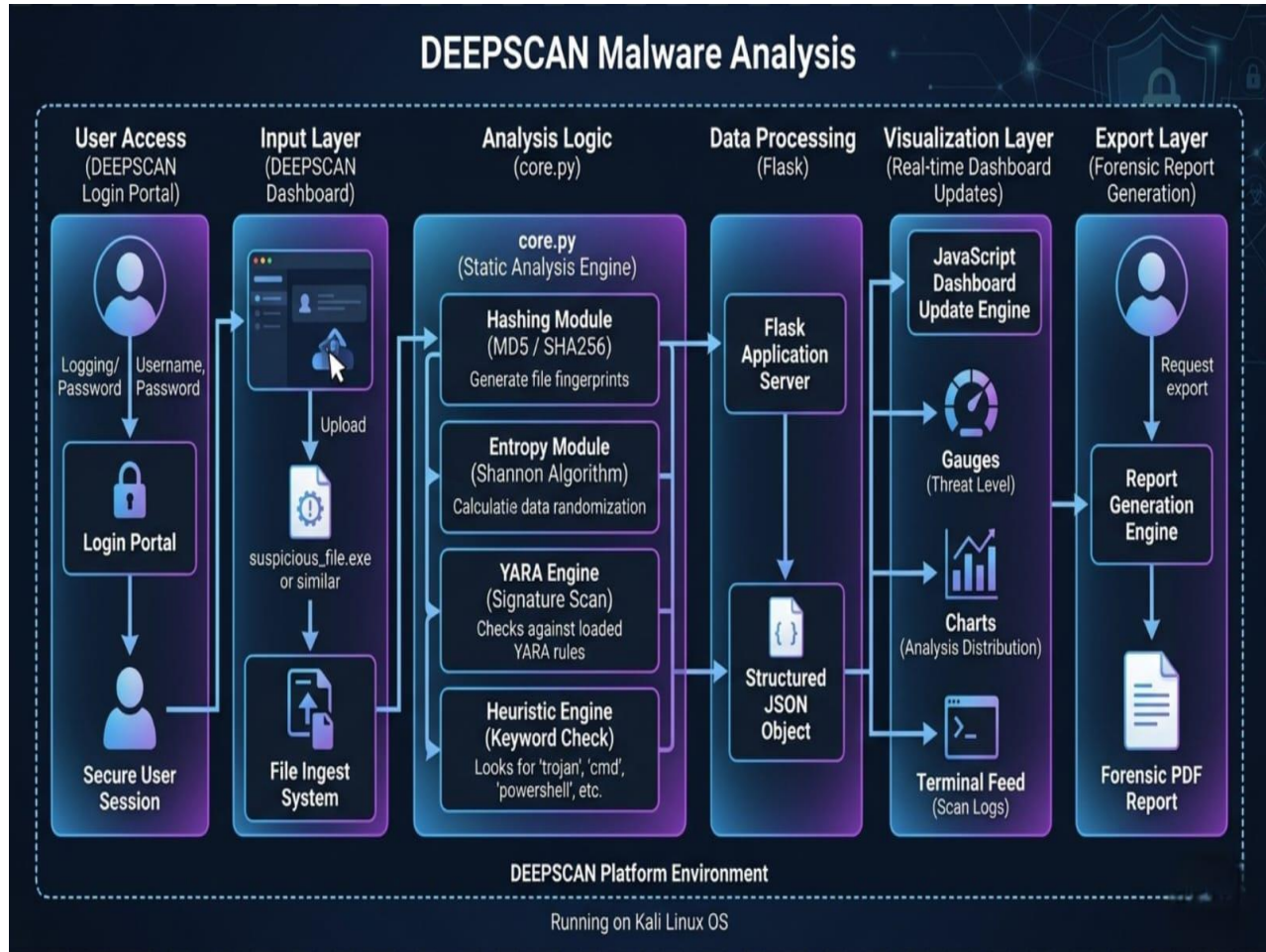
dashboard. This project addresses these issues by developing a Malware Detection and Analysis Dashboard using Flask and Python, providing an efficient and easy-to-use solution.

TOOLS USED

The development of this project involves several tools and technologies.

- **Python:** Python is used as the main programming language for developing the application logic, file processing, and analysis. It provides flexibility and strong library support.
- **Flask:** Flask is used as the web framework to build the backend of the application. It handles file uploads, processes requests, and connects different modules.
- **YARA:** YARA is used as the malware detection tool. It scans files using predefined rules and identifies suspicious patterns.
- **HTML, CSS, javascript:** These technologies are used to design the user interface and create an interactive dashboard for displaying results.
- **Reportlab:** reportlab is used to generate PDF reports of the scan results, including file details and analysis.
- **Python Libraries:** Libraries such as hashlib, mimetypes, and math are used for hash generation, file type detection, and entropy calculation.
- **Matplotlib:** Matplotlib is used for visualization of results through charts and graphs.
- **Visual Studio Code / Kali Linux Terminal:** Used for coding, testing, and running the project.
- **Kali Linux:** Used as the operating system for running cybersecurity tools and testing.
- **Python venv:** Used to manage project dependencies and maintain a clean environment.

ARCHITECTURE DIAGRAM OF THE PROJECT



EXISTING SYSTEM

The existing systems for malware detection mainly rely on traditional methods such as signature-based detection and standalone antivirus tools. These systems use predefined patterns to identify known malware and are widely implemented using tools like YARA. While these approaches are effective for detecting previously identified threats, they are limited in handling modern and evolving malware attacks.

Most existing solutions function as separate tools for scanning, analysis, or reporting, without integrating all features into a single platform. Additionally, many of these systems require technical expertise to operate and interpret results, making them less user-friendly for beginners.

DISADVANTAGES OF EXISTING SYSTEM

- Depends mainly on signature-based detection, which cannot identify new or unknown malware
- Ineffective against advanced malware techniques like obfuscation and polymorphism
- Lack of integrated analysis (no entropy calculation, hash analysis, etc.)
- Provides limited or raw output that is difficult to understand
- No proper visualization or dashboard for displaying results
- Requires technical knowledge to operate and interpret findings
- Separate tools needed for scanning, analysis, and report generation
- No real-time interaction or user-friendly interface
- Limited support for detailed reporting and documentation
- Cannot provide accurate risk classification of files

PROPOSED SYSTEM

The proposed system is a Malware Detection and Analysis Dashboard that integrates malware scanning, file analysis, and result visualization into a single platform. Unlike traditional systems, it combines signature-based detection using YARA with additional analytical techniques to improve accuracy and usability.

The system is developed using Flask and Python, providing a web-based interface where users can easily upload files and perform malware analysis. Once a file is uploaded, it is scanned using YARA rules to detect known malicious patterns. At the same time, the system performs detailed analysis by calculating entropy, identifying file type, generating hash values (MD5 and SHA256), and detecting suspicious features such as URLs or keywords.

Based on the results of scanning and analysis, the system calculates a risk score and classifies the file into categories such as Safe, Low Risk, Medium Risk, or High Risk. The dashboard presents all results in a structured and user-friendly format, including file summary, threat metrics, YARA rule matches, and detailed logs. Additionally, the system generates reports in text and PDF formats for documentation and further investigation.

ADVANTAGES OF PROPOSED SYSTEM

- Combines signature-based detection with advanced analysis techniques
- Improves detection accuracy compared to traditional systems
- Provides a user-friendly web dashboard for easy interaction
- Supports real-time file scanning and analysis
- Displays results in a clear and structured format
- Includes entropy calculation and hash-based analysis
- Detects suspicious features such as URLs and keywords
- Generates detailed reports in text and PDF formats
- Provides proper risk classification (Safe, Low, Medium, High)
- Reduces the need for multiple separate tools
- Requires minimal technical knowledge to use
- Scalable and can be extended with more rules and features

INPUT

app.py

```
from flask import Flask, render_template, request, jsonify, send_file, redirect, url_for, session
import os
from scripts.core import analyze
from scripts.report_pdf import generate_pdf

app = Flask(__name__)
app.secret_key = "secure_net_intelligence_key"

UPLOAD_FOLDER = "uploads"
OUTPUT_FOLDER = "output"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(OUTPUT_FOLDER, exist_ok=True)

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")

        # Simple Hardcoded Auth for demo
        if username == "admin" and password == "kali":
            session["logged_in"] = True
            return redirect(url_for("dashboard"))
        else:
            return render_template("login.html", error="Invalid Credentials")

    return render_template("login.html")

@app.route("/")
def dashboard():
    if not session.get("logged_in"):
        return redirect(url_for("login"))
    return render_template("dashboard.html")

@app.route("/logout")
def logout():
```



```

        session.clear()
        return redirect(url_for("login"))

@app.route("/scan", methods=["POST"])
def scan():
    if not session.get("logged_in"):
        return jsonify({"error": "Unauthorized"}), 401
    file = request.files["file"]
    path = os.path.join(UPLOAD_FOLDER, file.filename)
    file.save(path)

    data = analyze(path)
    return jsonify(data)

@app.route("/download_pdf", methods=["POST"])
def download_pdf():
    if not session.get("logged_in"):
        return redirect(url_for("login"))
    data = request.json["data"]
    generate_pdf(data)
    return send_file("output/report.pdf", as_attachment=True)

if __name__ == "__main__":
    app.run(debug=True, port=5000)

```

analyzer.py

```

import os
import mimetypes
import math

def calculate_entropy(file_path):
    with open(file_path, "rb") as f:
        data = f.read()

    if not data:
        return 0

    entropy = 0

```

```

    for x in range(256):
        p_x = data.count(bytes([x])) / len(data)
        if p_x > 0:
            entropy += - p_x * math.log2(p_x)

    return round(entropy, 2)

def analyze_file(file):
    size = os.path.getsize(file)
    file_type = mimetypes.guess_type(file)[0]
    entropy = calculate_entropy(file)

    return {
        "size": size,
        "type": file_type,
        "entropy": entropy
    }

```

core.py

```

import os, re, hashlib, mimetypes, math, subprocess

def calculate_entropy(file):
    with open(file, "rb") as f:
        data = f.read()

    if not data:
        return 0

    entropy = 0
    for x in range(256):
        p_x = data.count(bytes([x])) / len(data)
        if p_x > 0:
            entropy -= p_x * math.log2(p_x)

    return round(entropy, 2)

def real_file_type(file):
    with open(file, "rb") as f:

```

```

        header = f.read(4)
        if header.startswith(b"MZ"):
            return "Windows PE Executable"
        elif header.startswith(b"\x7fELF"):
            return "Linux ELF Binary"
        return "Unknown / Text"

def detect_features(file):
    features = []
    try:
        with open(file, "r", errors="ignore") as f:
            content = f.read().lower()

            if "http" in content:
                features.append("URL detected")
            if "password" in content:
                features.append("Sensitive keyword")
            if "malware" in content:
                features.append("Malware keyword")
            if "malicious" in content:
                features.append("Malicious behavior")
            if "attack" in content:
                features.append("Attack pattern")
            if "cmd" in content:
                features.append("Command execution")

    except:
        pass
    return features

def run_yara(file):
    result = subprocess.run(
        ["yara", "rules/rules.yar", file],
        capture_output=True,
        text=True
    )

    matches = result.stdout.strip().split("\n") if result.stdout else []

    return matches, result.stdout

```

```

def analyze(file):
    size = os.path.getsize(file)
    entropy = calculate_entropy(file)
    file_type = mimetypes.guess_type(file)[0]
    real_type = real_file_type(file)

    with open(file, "rb") as f:
        content_raw = f.read()
        content_text = content_raw.decode(errors="ignore").lower()

    md5 = hashlib.md5(content_raw).hexdigest()
    sha256 = hashlib.sha256(content_raw).hexdigest()

    matches, raw_yara = run_yara(file)
    features = detect_features(file)

    # 💧 IMPROVED RISK LOGIC
    # High: Hard keywords like 'trojan' or extreme entropy
    if "trojan" in content_text or entropy > 7.5 or "Basic_Malware_Detection" in
raw_yara:
        status = "HIGH"
    # Medium: High entropy or multiple suspicious features
    elif entropy > 6.0 or len(features) >= 2:
        status = "MEDIUM"
    # Low: Individual features like a URL
    elif len(features) > 0 or len(matches) > 0:
        status = "LOW"
    else:
        status = "SAFE"

    # Precise risk score calculation
    risk_score = min(100, (len(matches) * 25) + (len(features) * 15) + int(entropy
* 5))
    if "trojan" in content_text: risk_score = max(risk_score, 90)

    return {
        "file": os.path.basename(file),
        "size": size,
        "type": file_type,
        "real_type": real_type,

```

```

        "entropy": entropy,
        "md5": md5,
        "sha256": sha256,
        "features": features,
        "matches": matches,
        "match_count": len(matches),
        "risk": risk_score,
        "status": status,
        "log": raw_yara if raw_yara else f"Content Analysis found: {'',
'.join(features)}"
    }

```

dashboard.html

```

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Malware Detection Dashboard | Advanced Pro</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">
    <link rel="stylesheet" href="/static/style.css">
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>

<body>

    <!-- Sidebar -->
    <div class="sidebar">
        <i class="fas fa-shield-alt active"></i>
        <i class="fas fa-search"></i>
        <i class="fas fa-file-alt"></i>
        <i class="fas fa-cog"></i>
        <a href="/logout" style="margin-top: auto; margin-bottom: 25px; color:
var(--accent-red);"><i class="fas fa-power-off"></i></a>
    </div>

```

```

<div class="main-content">
  <header>
    <div class="title-row">
      <div>
        <h1 id="main_title">Malware Detection Dashboard</h1>
        <span class="subtitle">Advanced Malware Dashboard</span>
      </div>
      <div style="font-size: 0.8rem; color: var(--text-secondary);">
        <i class="fas fa-user-shield"></i> Analyst Mode
      </div>
    </div>

    <div class="file-search-bar">
      <i class="fas fa-folder-open" style="color: var(--accent-blue); margin-left: 10px;"></i>
      <input type="text" id="file_path_display" readonly placeholder="/media/sf_yara_project/samples/..." value="">
    </div>

    <div class="btn-group">
      <input type="file" id="fileInput" style="display: none;" onchange="updatePathDisplay()">
      <button class="btn btn-grey" onclick="document.getElementById('fileInput').click()">Browse</button>
      <button class="btn btn-green" onclick="scan()">Scan File</button>
      <button class="btn btn-green" onclick="savePDF()">Save Report</button>
    </div>
  </header>

  <div class="dashboard-container">
    <!-- Column 1: Metadata & YARA -->
    <div class="column">
      <div class="card">
        <div class="card-title">
          <i class="fas fa-info-circle"></i> File Summary & Metadata
        </div>
        <div class="metadata-list">
          <div class="metadata-item">
            <i class="fas fa-link"></i>
            <span class="metadata-label">Path:</span>
            <span class="metadata-value" id="res_path">--</span>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

        </div>
        <div class="metadata-item">
            <i class="fas fa-expand"></i>
            <span class="metadata-label">Size:</span>
            <span class="metadata-value" id="res_size">--</span>
        </div>
        <div class="metadata-item">
            <i class="fas fa-file-code"></i>
            <span class="metadata-label">Type:</span>
            <span class="metadata-value" id="res_type">--</span>
        </div>
        <div class="metadata-item">
            <i class="fas fa-microchip"></i>
            <span class="metadata-label">Real Type:</span>
            <span class="metadata-value" id="res_real">--</span>
        </div>
        <div class="metadata-item">
            <i class="fas fa-wave-square"></i>
            <span class="metadata-label">Entropy:</span>
            <div class="progress-container">
                <div class="progress-bar" id="entropy_bar"></div>
            </div>
        </div>
        <div class="metadata-item">
            <i class="fas fa-fingerprint"></i>
            <span class="metadata-label">MD5:</span>
            <span class="metadata-value" id="res_md5">--</span>
        </div>
        <div class="metadata-item">
            <i class="fas fa-fingerprint"></i>
            <span class="metadata-label">SHA256:</span>
            <span class="metadata-value" id="res_sha">--</span>
        </div>
    </div>
</div>

<div class="card yara-matches-card">
    <div class="card-title">
        <i class="fas fa-bug"></i> YARA RULE MATCHES
    </div>
    <div class="rule-group">
        <div>Rule
        Name:
        <span
id="yara_rules_detected">None</span></div>
    </div>

```

```

        </div>
    </div>

    <!-- Column 2: Risk & Metrics -->
    <div class="column">
        <div class="card">
            <div class="card-title">
                <i class="fas fa-exclamation-triangle"></i> Threat Metrics
& Risk
            </div>
            <div class="threat-grid">
                <div class="gauge-box">
                    <div class="main-gauge">
                        <div class="gauge-body"></div>
                        <div class="gauge-fill" id="risk_gauge"></div>
                        <div class="gauge-cover"></div>
                    </div>
                    <div class="gauge-value" id="risk_status">IDLE</div>
                    <div style="font-size: 0.9rem; color: var(--text-
secondary);">Risk Meter</div>
                </div>

                <div class="stat-card">
                    <div class="stat-label">Entropy Analysis</div>
                    <div class="gauge-box" style="padding: 5px;">
                        <div class="main-gauge" style="width: 140px;
height: 70px;">
                            <div class="gauge-body" style="width: 140px;
height: 140px;"></div>
                            <div class="gauge-fill" id="entropy_gauge"
                                style="width: 140px; height: 140px;
background: conic-gradient(from 180deg at 50% 50%, #2ecc71 0deg, #3498db 90deg,
#e74c3c 180deg);">
                                </div>
                                <div class="gauge-cover"
                                    style="width: 110px; height: 110px; top:
15px; left: 15px;"></div>
                            </div>
                            <div class="gauge-value" id="entropy_value_2"
                                style="font-size: 1.2rem; margin-top: 5px;
color: var(--accent-blue);">0.00</div>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>

```



```

        <!-- Bar Chart -->
        <div class="card" style="padding: 15px; margin-top:
10px;">
            <div class="stat-label" style="margin-bottom:
10px;">Threat Score Distribution</div>
            <canvas id="barChart"></canvas>
        </div>
    </div>
</div>

<!-- Column 3: Detailed Feed -->
<div class="column">
    <div class="card" style="margin-bottom: 20px;">
        <div class="card-title">
            <i class="fas fa-chart-pie"></i> Heuristics Breakdown
        </div>
        <canvas id="pieChart"></canvas>
    </div>

    <div class="card" style="flex: 1;">
        <div class="card-title">
            <i class="fas fa-list-ul"></i> Detailed Analysis Feed
        </div>
        <div class="feed-content" id="analysis_feed">
            [+] System initialized...
            [+] Awaiting file input for analysis...
        </div>
    </div>
</div>
</div>

<footer>
    <div class="status-indicator">
        <div class="dot"></div>
        System Status: <span style="color: var(--accent-green); margin-
left: 5px;">ONLINE</span>
    </div>
</div>

```

```

        YARA      Engine:      <span      style="color:      var(--accent-
blue);">v4.2.0</span>
      </div>
    </footer>
  </div>

  <script src="/static/script.js"></script>
</body>

</html>

```

login.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DEEPSCAN | Intelligence Suite</title>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">
  <link rel="stylesheet" href="/static/style.css">
  <style>
    .login-wrapper {
      height: 100vh;
      display: flex;
      align-items: center;
      justify-content: center;
      background: radial-gradient(circle at center, #1c2635 0%, #131b26
100%);
      width: 100%;
    }
    .login-card {
      width: 450px;
      padding: 40px;
      background: #1c2635;
      border-radius: 15px;
      border: 1px solid rgba(255, 255, 255, 0.05);
      box-shadow: 0 15px 50px rgba(0,0,0,0.5);
      text-align: center;
    }
    .login-card h2 {

```

```

        margin-bottom: 30px;
        font-size: 1.8rem;
        color: var(--accent-blue);
        letter-spacing: 2px;
    }
    .input-group {
        position: relative;
        margin-bottom: 25px;
    }
    .input-group i {
        position: absolute;
        left: 15px;
        top: 50%;
        transform: translateY(-50%);
        color: var(--text-secondary);
    }
    .input-group input {
        width: 100%;
        padding: 15px 15px 15px 45px;
        background: #0f1620;
        border: 1px solid var(--border);
        border-radius: 8px;
        color: white;
        font-size: 1rem;
        outline: none;
        transition: border-color 0.3s;
    }
    .input-group input:focus {
        border-color: var(--accent-blue);
    }
    .login-btn {
        width: 100%;
        padding: 15px;
        background: var(--accent-blue);
        color: white;
        border: none;
        border-radius: 8px;
        font-size: 1.1rem;
        font-weight: 700;
        cursor: pointer;
        transition: transform 0.2s, background 0.3s;
    }
    .login-btn:hover {
        background: #2980b9;
    }

```

```

        transform: translateY(-2px);
    }
    .error-msg {
        color: var(--accent-red);
        margin-bottom: 20px;
        font-size: 0.9rem;
    }
</style>
</head>
<body>
    <div class="login-wrapper">
        <div class="login-card">
            <div style="font-size: 3rem; color: var(--accent-blue); margin-bottom:
15px;">
                <i class="fas fa-user-shield"></i>
            </div>
            <h2>DEEPSCAN</h2>
            <p style="color: var(--text-secondary); margin-bottom: 30px; font-
size: 0.9rem;">ENTER CREDENTIALS FOR ADVANCED FILE ANALYTICS</p>

            {% if error %}
                <div class="error-msg">{{ error }}</div>
            {% endif %}

            <form action="/login" method="POST">
                <div class="input-group">
                    <i class="fas fa-user"></i>
                    <input type="text" name="username" placeholder="Username"
required autocomplete="off">
                </div>
                <div class="input-group">
                    <i class="fas fa-lock"></i>
                    <input type="password" name="password" placeholder="Password"
required>
                </div>
                <button type="submit" class="login-btn">AUTHENTICATE</button>
            </form>

            <p style="margin-top: 30px; font-size: 0.8rem; color: var(--text-
secondary);">
                SYSTEM LOG: Awaiting Authorization...
            </p>
        </div>
    </div>

```

```
    </div>
</body>
</html>
```

script.js

```
let lastData = null;
let barChart, pieChart;

// Initialize Charts on Load
window.onload = () => {
    const barCtx = document.getElementById('barChart').getContext('2d');
    barChart = new Chart(barCtx, {
        type: 'bar',
        data: {
            labels: ['File Size', 'Entropy', 'Heuristics', 'Signatures'],
            datasets: [{
                label: 'Threat Intensity',
                data: [0, 0, 0, 0],
                backgroundColor: ['#3498db', '#f1c40f', '#e67e22', '#e74c3c']
            }]
        },
        options: {
            responsive: true,
            scales: { y: { beginAtZero: true, max: 100 } },
            plugins: { legend: { display: false } }
        }
    });

    const pieCtx = document.getElementById('pieChart').getContext('2d');
    pieChart = new Chart(pieCtx, {
        type: 'pie',
        data: {
            labels: ['Safe', 'Suspicious', 'Critical'],
            datasets: [{
                data: [100, 0, 0],
                backgroundColor: ['#2ecc71', '#f1c40f', '#e74c3c']
            }]
        },
        options: { responsive: true }
    });
};
```

```

};

function updatePathDisplay() {
  const fileInput = document.getElementById('fileInput');
  const pathDisplay = document.getElementById('file_path_display');
  if (fileInput.files.length > 0) {
    pathDisplay.value = "/media/sf_yara_project/samples/" +
fileInput.files[0].name;
  }
}

function addLog(text, color = "var(--text-secondary)") {
  const feed = document.getElementById("analysis_feed");
  const line = document.createElement("div");
  line.className = "feed-line";
  line.style.color = color;
  line.innerText = text;
  feed.appendChild(line);
  feed.scrollTop = feed.scrollHeight;
}

function scan() {
  const fileInput = document.getElementById("fileInput");
  if (fileInput.files.length === 0) {
    alert("Please select a file first!");
    return;
  }

  const file = fileInput.files[0];
  const formData = new FormData();
  formData.append("file", file);

  const scanBtn = document.querySelector(".btn-green:nth-of-type(2)");
  scanBtn.innerText = "Scanning...";
  scanBtn.disabled = true;

  addLog(`[+] Initiating analysis for: ${file.name}`, "var(--accent-blue)");

  fetch("/scan", {
    method: "POST",
    body: formData
  })
  .then(res => res.json())

```

```

.then(data => {
  lastData = data; // Store full data for PDF
  // Update Metadata
  document.getElementById("res_path").innerText =
"/media/sf_yara_project/samples/" + data.file;
  document.getElementById("res_size").innerText = data.size + " bytes";
  document.getElementById("res_type").innerText = data.type || "Unknown";
  document.getElementById("res_real").innerText = data.real_type;
  document.getElementById("res_md5").innerText = data.md5;
  document.getElementById("res_sha").innerText = data.sha256.substring(0,
32) + "...";

  // Update Entropy Metadata Bar
  const entropyPercent = (data.entropy / 8) * 100;
  document.getElementById("entropy_bar").style.width = entropyPercent + "%";

  // Update New Entropy Gauge
  const entropyAngle = (data.entropy / 8) * 180;
  document.getElementById("entropy_gauge").style.transform =
`rotate(${entropyAngle}deg)`;
  document.getElementById("entropy_value_2").innerText =
data.entropy.toFixed(2);

  // Dynamic Color for Entropy Value
  const entropyColor = data.entropy > 7 ? "var(--accent-red)" : data.entropy
> 5 ? "var(--accent-yellow)" : "var(--accent-blue)";
  document.getElementById("entropy_value_2").style.color = entropyColor;

  // Update Risk Gauge
  const angle = (data.risk / 100) * 180;
  document.getElementById("risk_gauge").style.transform =
`rotate(${angle}deg)`;
  document.getElementById("risk_status").innerText = data.status;
  document.getElementById("risk_status").style.color =
    data.status === "HIGH" ? "var(--accent-red)" :
    data.status === "MEDIUM" ? "var(--accent-yellow)" : "var(--accent-
green)";

  // Update YARA Rules Display
  const yaraText = data.matches.length > 0 ? data.matches.join(", ") : "None
Detected";

```

```

        document.getElementById("yara_rules_detected").innerText = yaraText;
        document.getElementById("yara_rules_detected").style.color =
data.matches.length > 0 ? "var(--accent-red)" : "var(--text-primary)";

        // Update Charts
        if (barChart && pieChart) {
            barChart.data.datasets[0].data = [
                Math.min(100, data.size / 1000), // Normalized size
                entropyPercent,
                data.features.length * 20,
                data.match_count * 25
            ];
            barChart.update();

            const pieData = data.status === "HIGH" ? [10, 20, 70] : data.status
=== "MEDIUM" ? [30, 50, 20] : [90, 10, 0];
            pieChart.data.datasets[0].data = pieData;
            pieChart.update();
        }

        // Feed Logs
        addLog(`[+] Filesize: ${data.size} bytes`);
        addLog(`[+] Entropy detected: ${data.entropy}`);
        if (data.matches.length > 0) {
            addLog(`[!] ALERT: ${data.match_count} Signatures Matched!`, "var(--
accent-red)");
        }
        addLog(`[+] Security Status: ${data.status}`, data.status === "SAFE" ?
"var(--accent-green)" : "var(--accent-red)");

        scanBtn.innerText = "Scan File";
        scanBtn.disabled = false;
    })
    .catch(err => {
        addLog(`[!] Error during scan: ${err.message}`, "var(--accent-red)");
        scanBtn.innerText = "Scan File";
        scanBtn.disabled = false;
    });
}

function savePDF() {
    if (!lastData) {

```



```

        alert("Please run a scan first!");
        return;
    }
    fetch("/download_pdf", {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({data: lastData})
    })
    .then(res => res.blob())
    .then(blob => {
        const a = document.createElement("a");
        a.href = URL.createObjectURL(blob);
        a.download = `Malware_Analysis_${lastData.file}.pdf`;
        a.click();
    });
}

```

style.css

```

@import
url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Fira+Code:wght@400;500&display=swap');

:root {
    --bg-main: #131b26;
    --bg-card: #1c2635;
    --sidebar-bg: #0f1620;
    --accent-blue: #3498db;
    --accent-green: #2ecc71;
    --accent-yellow: #f1c40f;
    --accent-red: #e74c3c;
    --text-primary: #ecf0f1;
    --text-secondary: #bdc3c7;
    --border: rgba(255, 255, 255, 0.05);
}

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Inter', sans-serif;
}

```

```

html {
  font-size: 18px; /* Globally bigger font */
}

body {
  background-color: var(--bg-main);
  color: var(--text-primary);
  display: flex;
  height: 100vh;
  overflow: hidden;
}

/* Sidebar */
.sidebar {
  width: 70px;
  background: var(--sidebar-bg);
  display: flex;
  flex-direction: column;
  align-items: center;
  padding-top: 25px;
  gap: 35px;
  border-right: 1px solid var(--border);
}

.sidebar i {
  color: var(--text-secondary);
  font-size: 1.4rem;
  cursor: pointer;
  transition: color 0.3s;
}

.sidebar i:hover, .sidebar i.active {
  color: var(--accent-blue);
}

/* Main Content */
.main-content {
  flex: 1;
  display: flex;
  flex-direction: column;
  overflow-y: auto;
}

```

```

header {
  padding: 20px 30px;
  background: var(--bg-card);
  border-bottom: 2px solid var(--border);
}

.title-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
}

.title-row h1 {
  font-size: 1.8rem;
  font-weight: 700;
}

.subtitle {
  font-size: 0.9rem;
  color: var(--text-secondary);
  text-transform: uppercase;
  letter-spacing: 2px;
}

.file-search-bar {
  display: flex;
  gap: 12px;
  background: #0f1620;
  padding: 12px;
  border-radius: 8px;
  align-items: center;
}

.file-search-bar input {
  flex: 1;
  background: transparent;
  border: none;
  color: var(--text-primary);
  font-family: 'Fira Code', monospace;
  font-size: 1rem;
  padding: 5px 10px;
}

```

```

    outline: none;
}

.btn-group {
  display: flex;
  gap: 15px;
  margin-top: 20px;
  justify-content: center;
}

.btn {
  padding: 12px 24px;
  border-radius: 6px;
  font-weight: 600;
  cursor: pointer;
  border: none;
  font-size: 1rem;
  transition: transform 0.2s, opacity 0.2s;
}

.btn:hover { transform: translateY(-2px); opacity: 0.9; }
.btn-grey { background: #4a5568; color: white; }
.btn-green { background: #38a169; color: white; }

/* Dashboard Grid */
.dashboard-container {
  padding: 25px;
  display: grid;
  grid-template-columns: 1.2fr 1.2fr 1.6fr;
  gap: 25px;
  flex: 1;
}

.column {
  display: flex;
  flex-direction: column;
  gap: 25px;
}

.card {
  background: var(--bg-card);
  border-radius: 12px;
  border: 1px solid var(--border);
}

```

```

padding: 25px;
box-shadow: 0 8px 30px rgba(0,0,0,0.3);
}

.card-title {
  font-size: 1.1rem;
  font-weight: 700;
  color: var(--text-primary);
  margin-bottom: 25px;
  display: flex;
  align-items: center;
  gap: 12px;
}

.metadata-list {
  display: flex;
  flex-direction: column;
  gap: 15px;
}

.metadata-item {
  display: flex;
  align-items: center;
  gap: 15px;
  font-size: 1rem;
}

.metadata-label {
  color: var(--text-secondary);
  min-width: 100px;
  font-weight: 500;
}

.metadata-value {
  color: var(--text-primary);
  word-break: break-all;
  font-weight: 400;
}

.progress-container {
  height: 12px;
  background: #0f1620;
  border-radius: 6px;
}

```

```

    flex: 1;
    overflow: hidden;
    border: 1px solid var(--border);
}

.progress-bar {
    height: 100%;
    background: var(--accent-blue);
    width: 0%;
    transition: width 0.8s cubic-bezier(0.175, 0.885, 0.32, 1.275);
}

/* YARA Matches */
.yara-matches-card {
    flex: 1;
}

.rule-group {
    margin-bottom: 15px;
    font-size: 1rem;
}

/* Threat & Risk */
.threat-grid {
    display: flex;
    flex-direction: column;
    gap: 20px;
}

.gauge-box {
    text-align: center;
    padding: 15px;
}

.main-gauge {
    width: 220px;
    height: 110px;
    margin: 0 auto;
    position: relative;
    overflow: hidden;
}

.gauge-body {

```

```

width: 220px;
height: 220px;
border-radius: 50%;
background: #0f1620;
position: absolute;
top: 0;
left: 0;
}

.gauge-fill {
width: 220px;
height: 220px;
border-radius: 50%;
background: conic-gradient(from 180deg at 50% 50%, var(--accent-green) 0deg,
var(--accent-yellow) 90deg, var(--accent-red) 180deg);
position: absolute;
top: 0;
left: 0;
transform-origin: center;
transform: rotate(0deg);
transition: transform 1.5s cubic-bezier(0.4, 0, 0.2, 1);
}

.gauge-cover {
width: 180px;
height: 180px;
background: var(--bg-card);
border-radius: 50%;
position: absolute;
top: 20px;
left: 20px;
}

.gauge-value {
font-size: 2rem;
font-weight: 800;
margin-top: 15px;
}

.stat-card {
background: rgba(15, 22, 32, 0.5);
padding: 15px;
border-radius: 8px;

```

```

    display: flex;
    flex-direction: column;
    gap: 10px;
}

.stat-label {
    font-size: 0.9rem;
    color: var(--text-secondary);
    font-weight: 500;
}

.stat-value {
    font-weight: 700;
    font-size: 1.1rem;
}

/* Chart Styles */
.chart-container {
    height: 250px;
    width: 100%;
    margin-top: 15px;
}

/* Feed */
.feed-content {
    font-family: 'Fira Code', monospace;
    font-size: 0.9rem;
    color: var(--text-secondary);
    white-space: pre-wrap;
    height: 500px;
    overflow-y: auto;
    padding: 10px;
    background: rgba(0,0,0,0.2);
    border-radius: 8px;
}

.feed-line {
    margin-bottom: 6px;
    line-height: 1.4;
}

/* Footer */
footer {

```

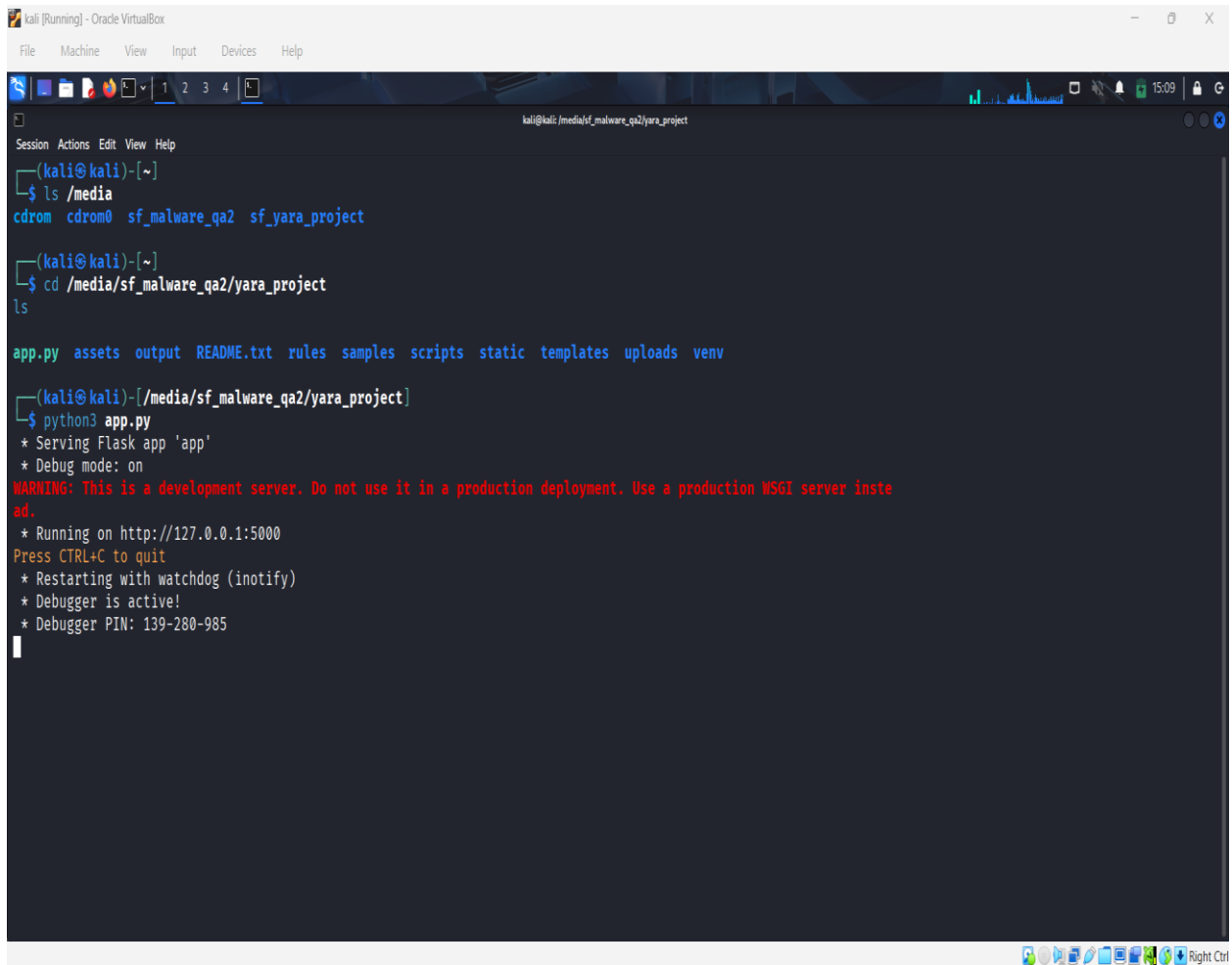


```
padding: 15px 30px;
background: var(--bg-card);
border-top: 2px solid var(--border);
display: flex;
justify-content: space-between;
font-size: 0.9rem;
color: var(--text-secondary);
}

.status-indicator {
  display: flex;
  align-items: center;
  gap: 10px;
}

.dot {
  width: 10px;
  height: 10px;
  background: var(--accent-green);
  border-radius: 50%;
  box-shadow: 0 0 15px var(--accent-green);
}

::-webkit-scrollbar { width: 8px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb { background: #2c3e50; border-radius: 10px; }
```



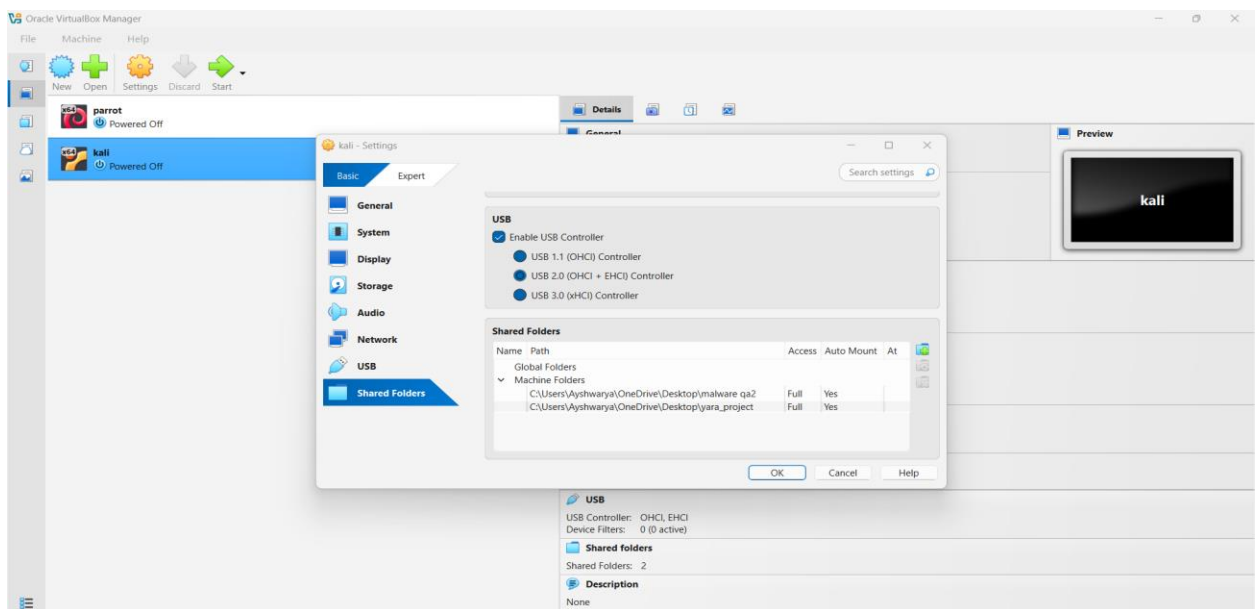
```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali:/media/sf_malware_qa2/yara_project
Session Actions Edit View Help
(kali@kali)-[~]
$ ls /media
cdrom cdrom0 sf_malware_qa2 sf_yara_project

(kali@kali)-[~]
$ cd /media/sf_malware_qa2/yara_project
ls

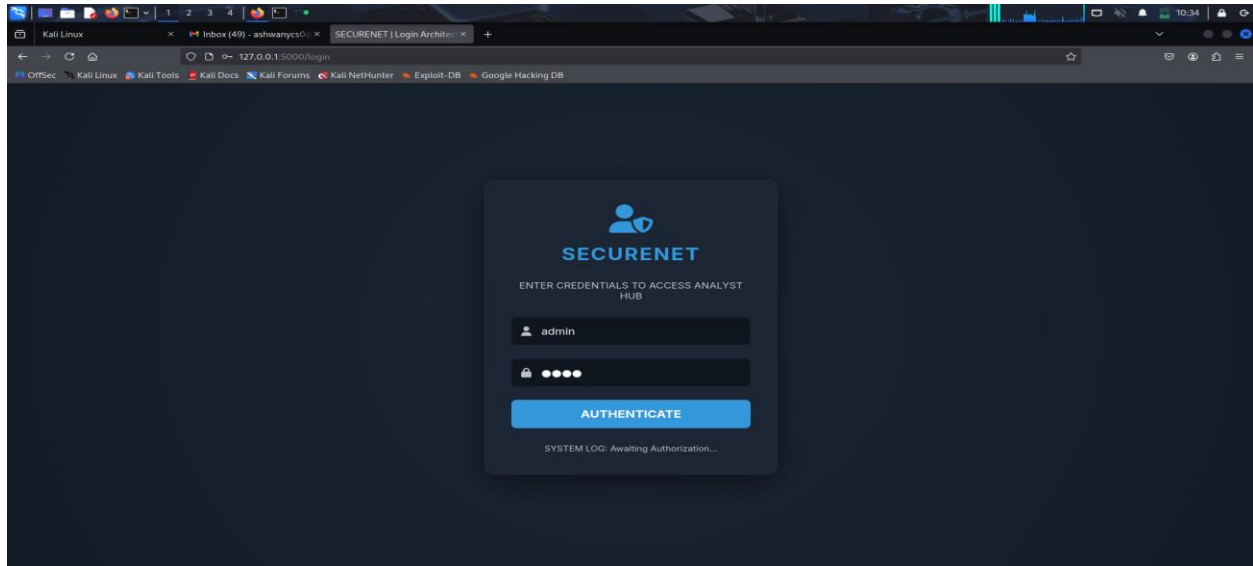
app.py assets output README.txt rules samples scripts static templates uploads venv

(kali@kali)-[/media/sf_malware_qa2/yara_project]
$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (inotify)
* Debugger is active!
* Debugger PIN: 139-280-985
```

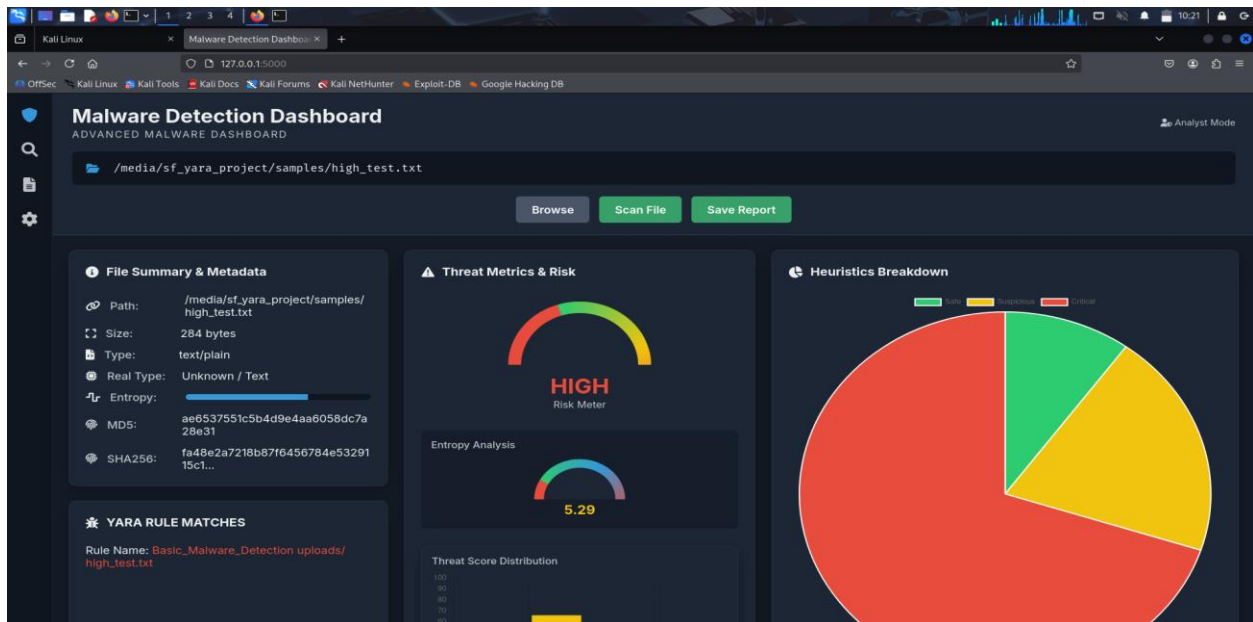


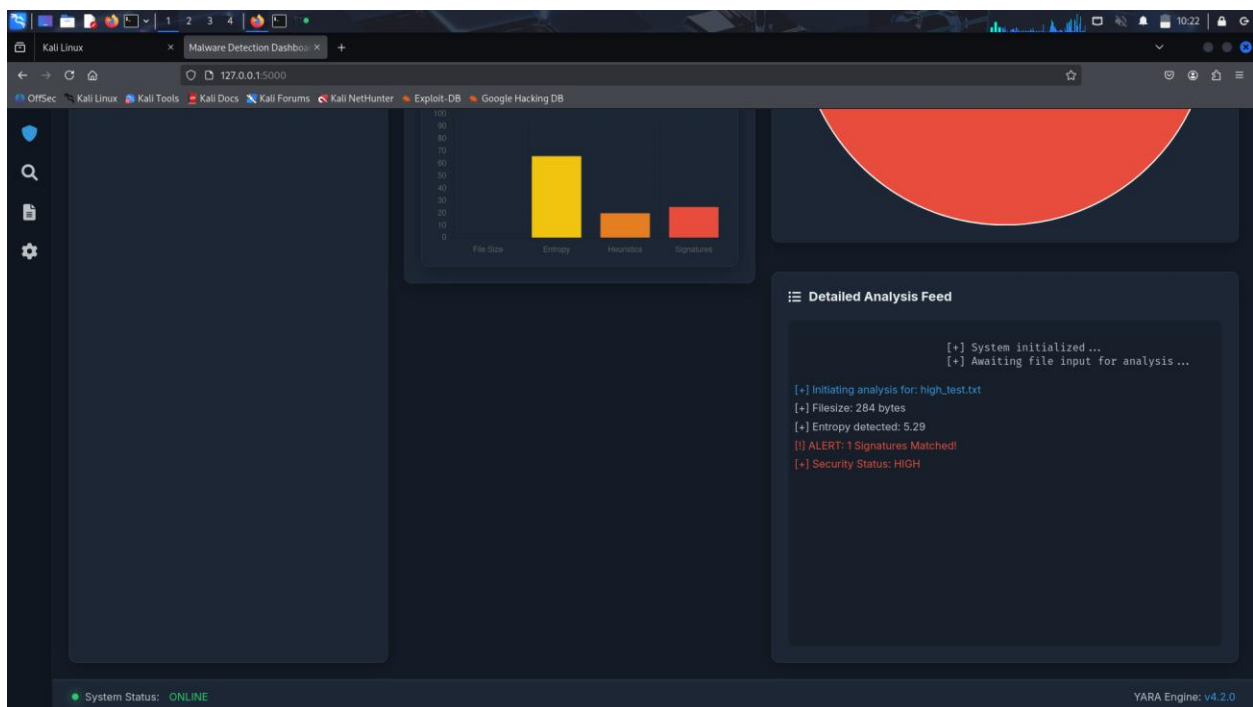
OUTPUT

Login Page

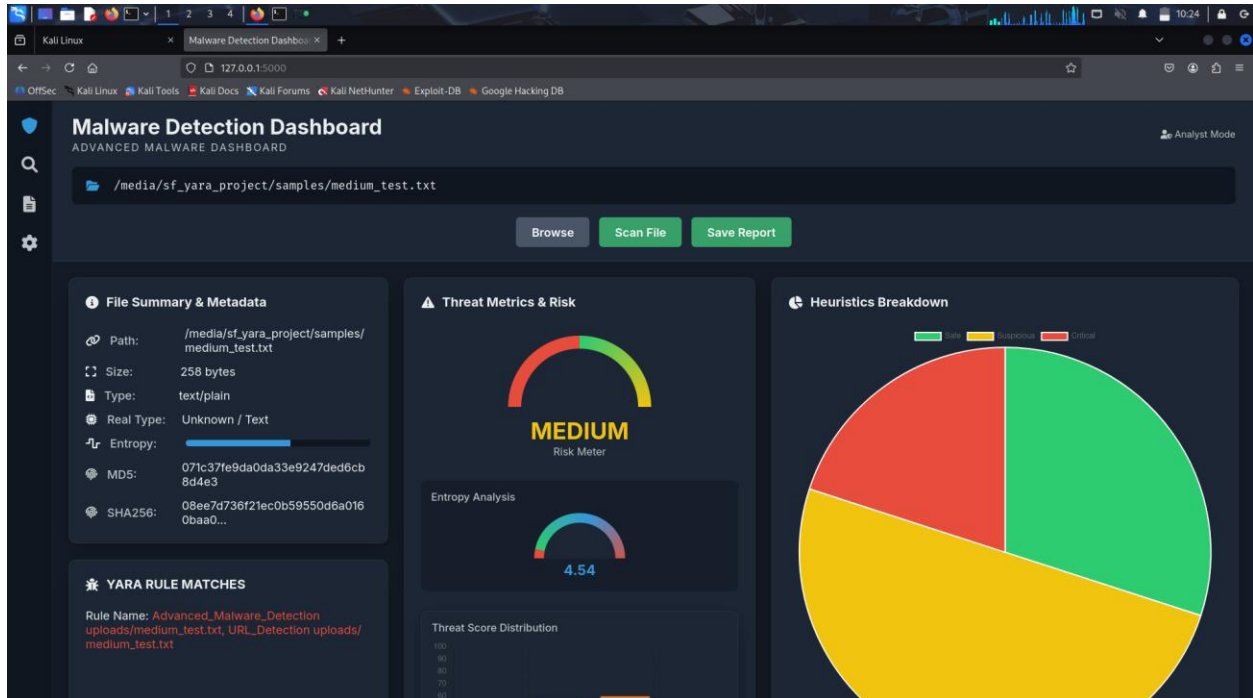


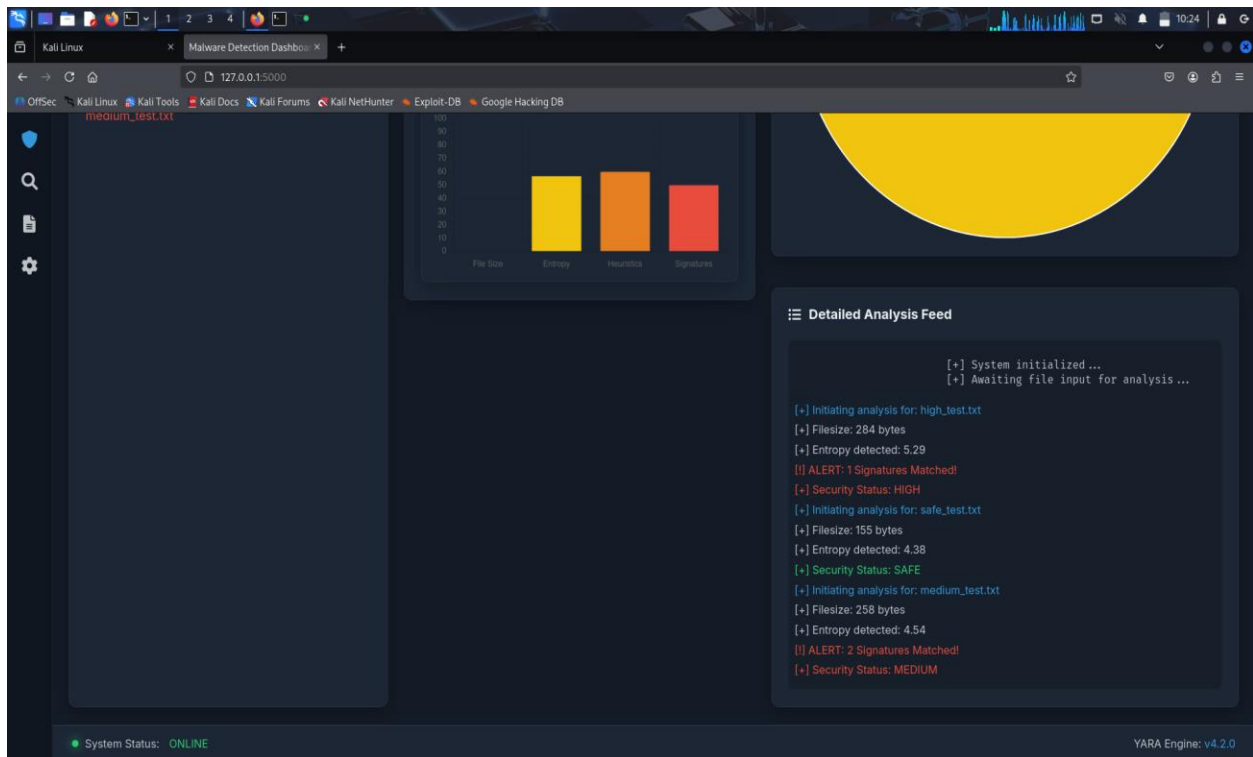
High Risk File Analysis



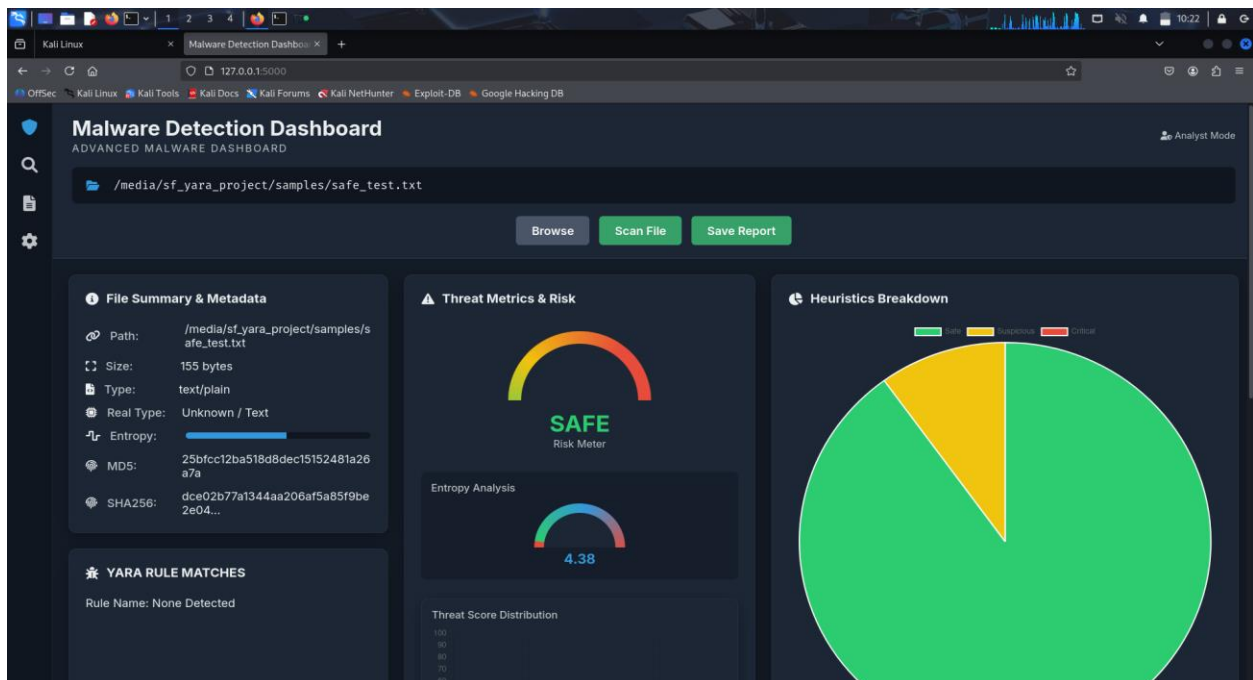


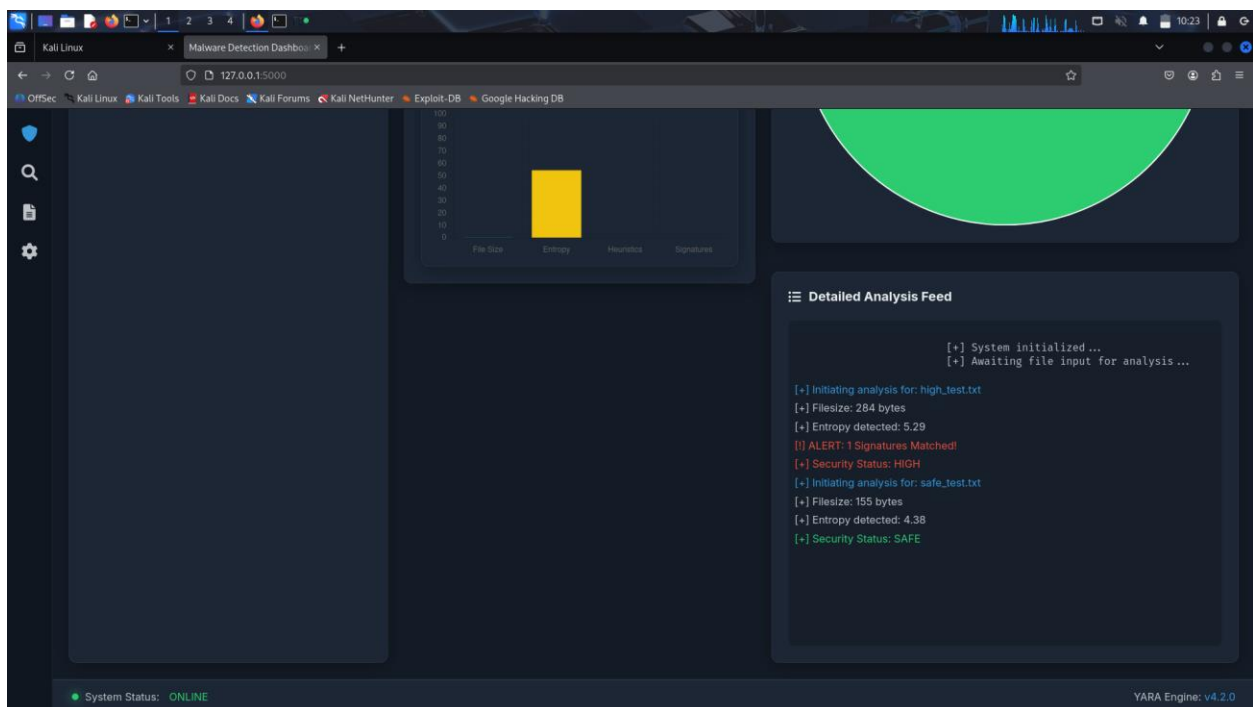
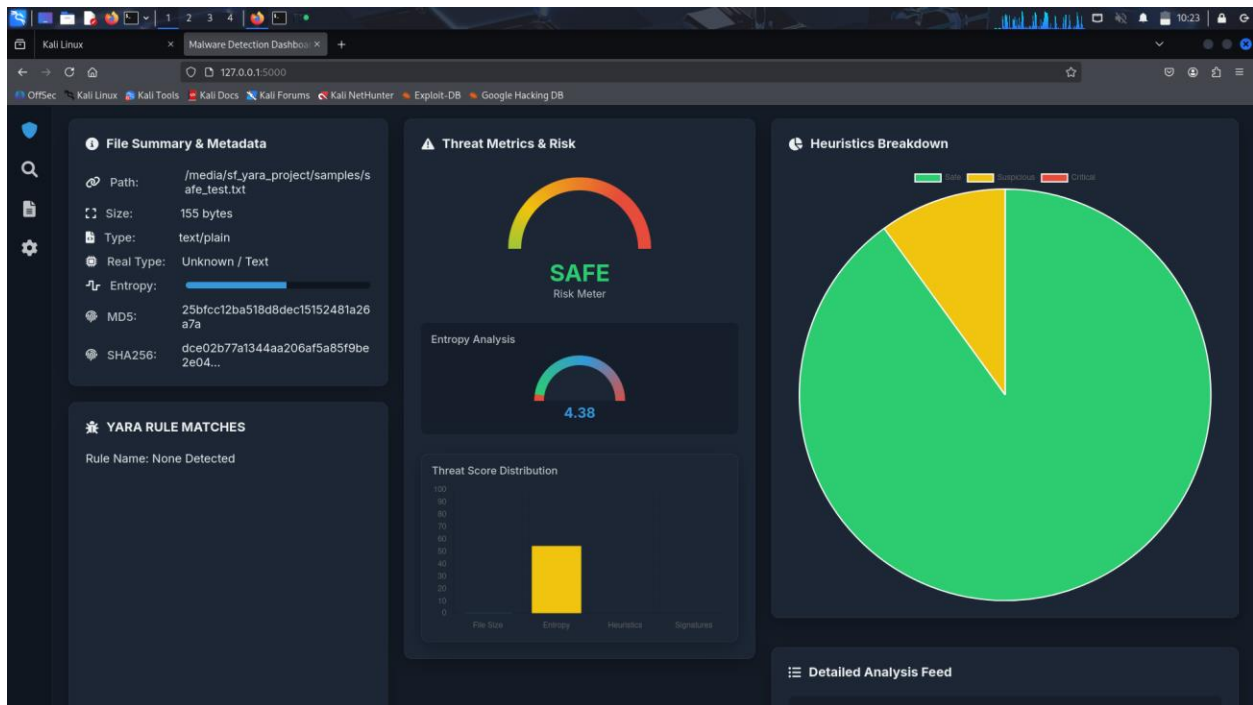
Medium Risk File Analysis



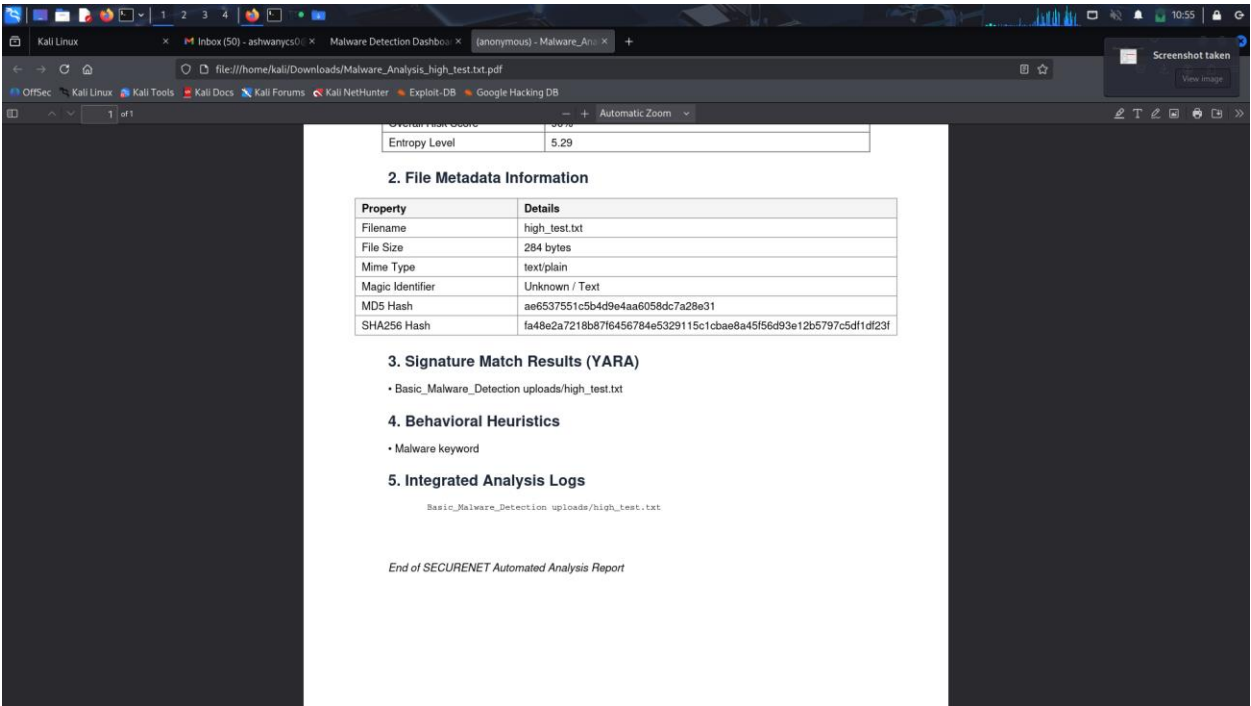
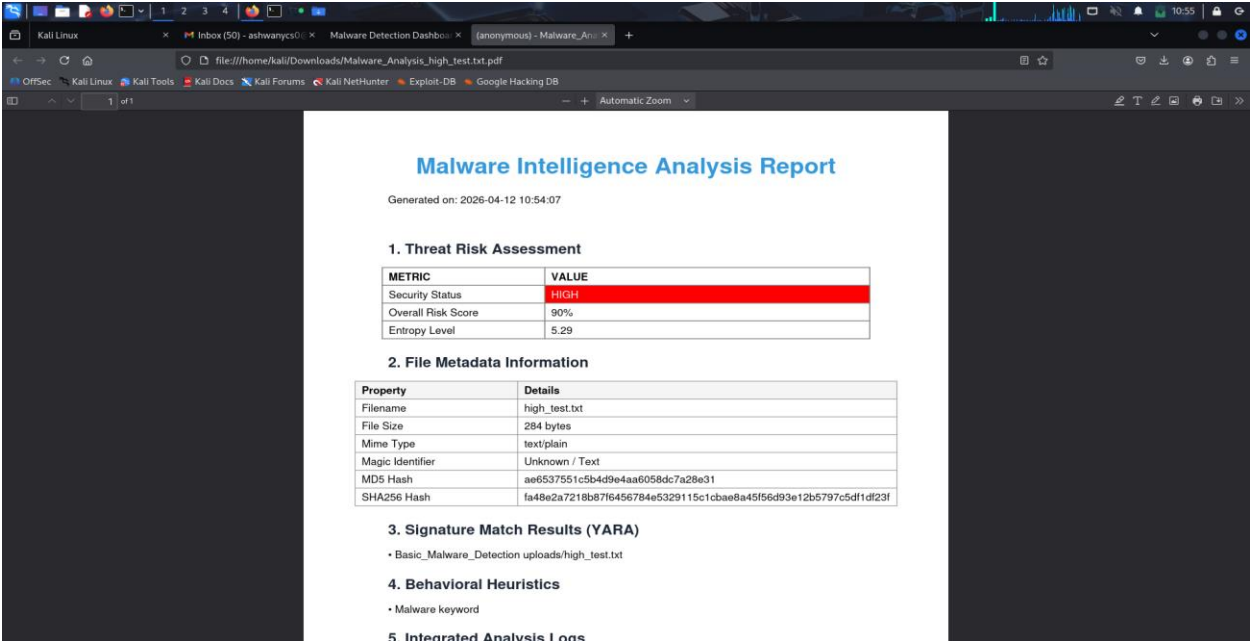


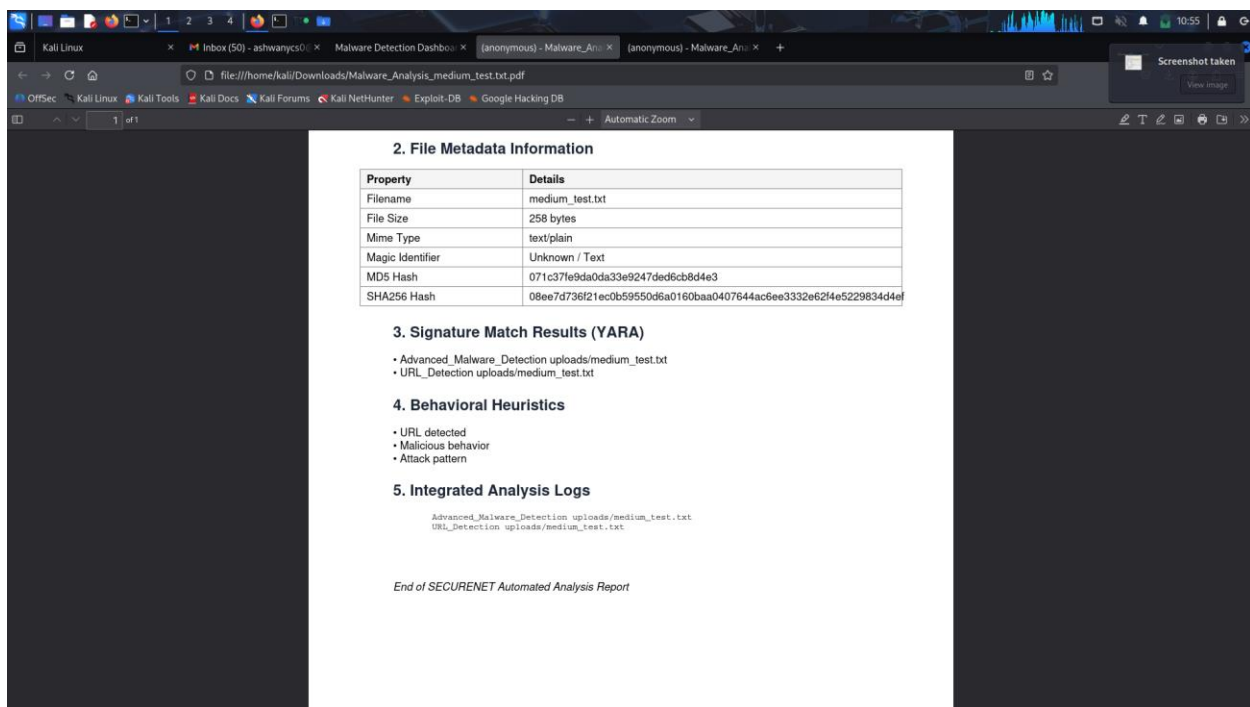
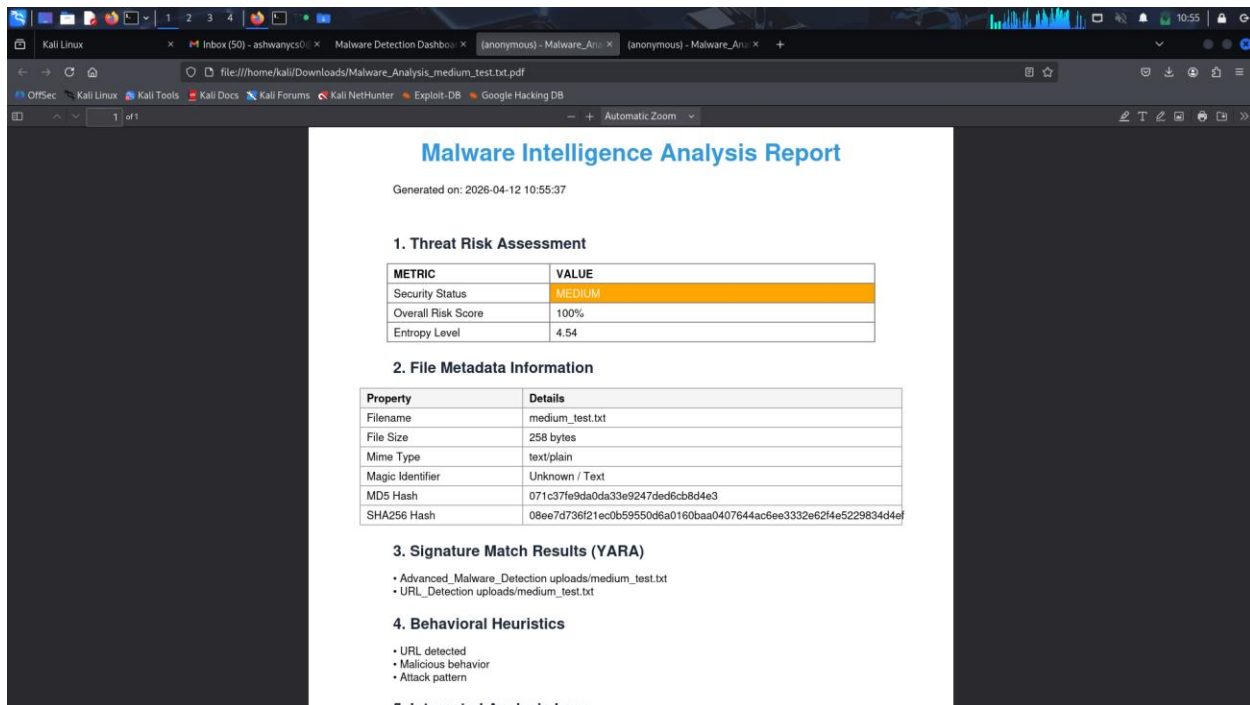
Safe File Analysis

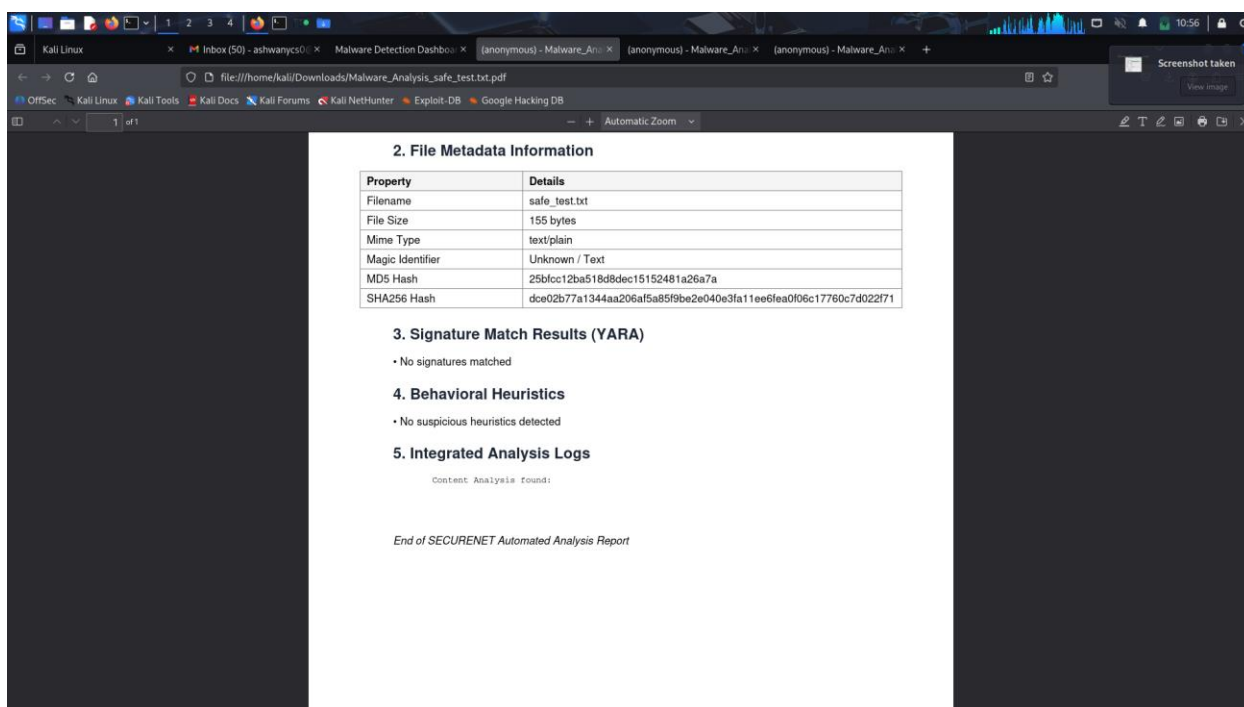
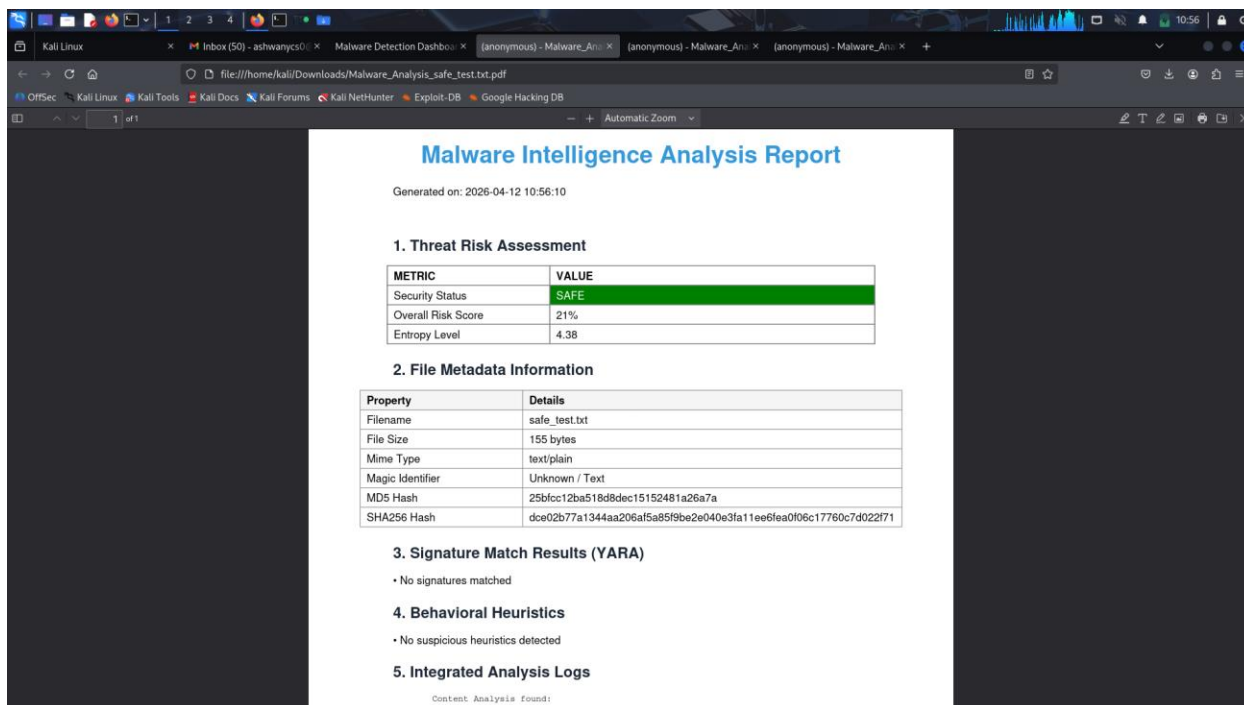




PDF Reports Generated







FUTURE RECOMMENDATIONS

The project can be further enhanced by adding advanced features to improve functionality and usability

- Machine learning techniques can be integrated to detect unknown and zero-day malware more effectively.
- The YARA rules database can be expanded to improve detection accuracy and coverage.
- Real-time monitoring can be added to continuously scan files and system activities.
- Cloud-based storage can be implemented for handling large-scale file analysis.
- User authentication and role-based access control can be added to improve system security.
- Automated alerts such as email or notifications can be enabled for high-risk detections.
- Behavioral analysis using sandbox execution can be included for advanced threat detection.
- Multi-file and batch processing can be supported for faster and efficient analysis.